PanG2G: Scalable Base-Pair-Resolution Graph-to-Graph Alignment of Pangenome Graphs

Ahmad Shakleya and Suhaib A. Fahmy
King Abdullah University of Science and Technology, Thuwal, Saudi Arabia.
ahmad.shakleya@kaust.edu.sa

***Abstract*—Pangenome variation graphs have replaced the single linear reference as the state of the art for representing human genetic diversity, yet a basic operation is still missing: directly comparing two such graphs to each other. On population pangenomes the delta graph shows the population-specific structural variation that a sequence-to-reference method cannot express without a linear projection. We present PanG2G, the first system for base-pair resolution graph-to-graph alignment of pangenome variation graphs. In this work we characterize the workload, describe the system and validate the reference-anchored output on a planted ground truth. The reference-anchored diff scales near-linearly to an 8.2×10^9 bp graph pair ($\sim2.6\times$ the human genome) on a single core in 72 min with 100% precision/recall, and the parallel snarl-pairing engine reaches $50.1\times$ DP-phase speedup on 64 cores.**

***Index Terms*—comparative genomics, pangenome, variation graph, graph alignment, structural variation**

I. Introduction

Genome analysis has long relied on a single linear reference, which introduces a systematic bias: alleles absent from the reference are hard to detect [1], [2]. Pangenome variation graphs remove this bias by encoding many haplotypes as paths through a graph whose bubbles make polymorphism explicit [3]: the Human Pangenome Reference Consortium (HPRC) draft, built from 47 diploid samples, demonstrates this approach at scale [4].

The availability of multiple population-specific pangenomes raises a natural question: how do two such graphs differ, and which alleles are private to one? Answering this requires aligning one graph to another. Sequence-to-sequence [5], [6] and sequence-to-graph alignment [1], [7], [8] are well established, but no tool performs graph-to-graph alignment at base-pair resolution. In practice, comparing two pangenomes still means projecting both onto a common linear reference, reintroducing the very bias pangenomes were built to remove [4], [9].

In general, graph-to-graph alignment contains subgraph isomorphism and is therefore NP-hard [10]. The key observation is that variation graphs are not arbitrary graphs. They decompose into a hierarchy of snarls, the generalized bubbles of Paten et al. [11], and this snarl tree has bounded tree-width in practice [12].

We exploit this observation to build PanG2G, a system that computes these graph-to-graph aligments efficiently. The workload is irregular: most snarls are tiny single-nucleotide polymorphism (SNP) bubbles, but a few structural-variant regions are enormous, so per-snarl work is heavy-tailed and a static work partition is load-imbalanced. PanG2G addresses this with three systems techniques: a cache-aware memory layout for the snarl tree, a minimizer index for anchor detection, and a priority-dispatch scheduler keyed on estimated subtree work, each motivated by direct profiling of the workload.

PanG2G factors the alignment into independent per-snarl bipartite-matching subproblems, each solved optimally in $O(k^3)$ by the linear assignment problem solver using Jonker-Volgenant (LAPJV): given the $k \times k$ matrix of pairwise allele dissimilarities, it finds the one-to-one matching between the two snarls' alleles with minimum total cost, computed via shortest augmenting paths [13]. Snarl pairing, however, is greedy, so end-to-end optimality is not guaranteed.

The contributions made in this paper include:

- **A problem definition and working system.** We define base-pair-resolution graph-to-graph alignment for variation graphs and present PanG2G, the first system to solve it. The delta graph it outputs is the graph-level analogue of a VCF: every allele present in one population graph but absent from the other.
- **A workload characterization that drives the design.** Using the reference prototype we measure where time and memory go and show per-snarl work is heavy-tailed (Gini > 0.55) and scale-invariant.
- **A parallel, cache-aware implementation.** A C++20 engine with a van Emde Boas snarl-tree layout, minimizer anchor index, exact LAPJV bipartite matching, and an OpenMP priority-dispatch runtime.
- **Evaluation on synthetic and real pangenomes,** including a reference-anchored diff that scales near-linearly past whole-genome size, and real African vs. East Asian (1000 Genomes) population graphs whose delta is significantly enriched near gnomAD-SV and ClinVar pathogenic sites ($p < 10^{-3}$), demonstrating the delta lands on real structural and clinical variation.

II. Related Work

The earliest alignment algorithms compared two sequences with dynamic programming: Needleman-Wunsch [5] for long read mapping and Smith-Waterman [6] for short read mapping, both $O(mn)$. The short-read era added Burrows-Wheeler-based mappers such as BWA [14] and Bowtie 2 [15], and the long-read era brought minimap2 [16], which seeds alignments with minimizers [17], a sketching idea we reuse for anchor

detection. All of these are sequence-to-sequence mappers: both inputs are strings.

The move to pangenomes made the reference a graph while the query stayed a sequence. The vg toolkit established variation graphs as a data model [18]. GraphAligner aligns long reads to graphs [19]. PanGenie genotypes from a pangenome by k-mer matching [20], and ParSGA gives a work-optimal parallel algorithm for aligning sequences to genome graphs [21]. Graph construction itself is mature: minigraph builds SV-scale graphs [22], Minigraph-Cactus produces base-level HPRC graphs [23], [24], and the HPRC has released reference-quality graphs [4]. Every tool in this paragraph shares one limitation: the query is a sequence. None aligns two graphs.

A few tools compare graphs, but not at base-pair variation resolution. PanGraph builds and compares bacterial pangenome graphs at the level of homologous gene blocks [25], and SplitMEM relates pangenome representations via suffix structures [26]. These operate on gene-block granularity, not on the individual alleles of a variation graph. The community has recognized the gap: the Pangenome Graph Variation Format (PGVF) [27], a GFA fork, has been proposed to represent graph-to-graph alignments, but no tool computes them. PanG2G is, to our knowledge, the first system that does so at allele resolution.

III. Methodology

A. The Algorithm

PanG2G consists of five stages (Fig. 1): (1) detect anchors, backbone nodes shared by both graphs, to pin the two coordinate systems together; (2) decompose each graph into its snarl tree; (3) pair snarls across the graphs; (4) run a per-snarl-pair dynamic program whose leaf operation is allele matching; (5) emit the delta graphs $\Delta_{A\to B}, \Delta_{B\to A}$, a shared graph, and a global score (Fig. 2).

The formulation and system support the general nested recurrence (2), but the graphs we evaluate (vg construct and synthetic generators) contain only depth-1 snarls: simple SNP/indel bubbles with no nested children. So the nested recurrence, the van Emde Boas layout, and the directed acyclic graph (DAG) dependencies are exercised only at depth 1, where they reduce to independent per-snarl-pair assignments. They are retained because a full Minigraph-Cactus backend's nested snarls require them, but the ablations and scaling numbers in §IV should be read with this depth-1 scope in mind.

Given graphs G_A, G_B and a matching $\mathscr{M} = (M_v, M_e, M_s)$ of their nodes, edges, and snarls, with σ_A, σ_B the node sequence labels in G_A, G_B, the alignment score is

$$W(G_A, G_B, \mathscr{M}) = \alpha \sum_{(u,v)\in M_v} \mathrm{sim}(\sigma_A(u), \sigma_B(v)) + \beta \sum_{(e,e')\in M_e} \mathbf{1}[\text{cons.}] + \gamma \sum_{(S,S')\in M_s} \mathrm{SimBubble}(S, S'), \quad (1)$$

where $\mathrm{sim}(s_1, s_2) = 1 - \mathrm{edit}(s_1, s_2)/\max(|s_1|, |s_2|)$ and (α, β, γ) weight the terms (defaults $1, 0.5, 2$; the structural term is up-weighted because allele identity is the primary delta signal). We do not sweep (α, β, γ): the delta depends only on the within-snarl matching μ, set by γ, and the ranking of allele pairs is invariant to γ's global scale. Since sibling snarls are separated by their boundary nodes, their subproblems are independent. At depth-1 each snarl pair is a leaf, reducing to one $O(k^3)$ bipartite matching per pair.

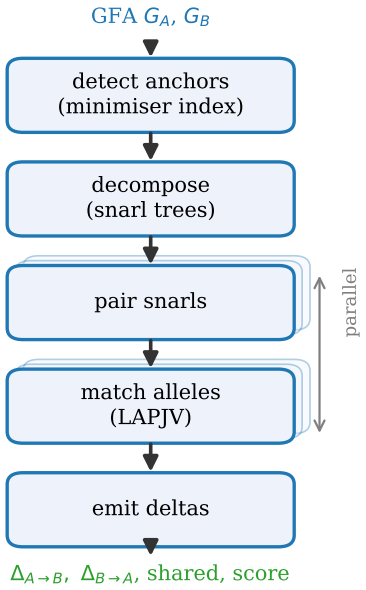


Fig. 1. The PanG2G pipeline. (1) Detect anchors via a minimizer index to pin the two coordinate systems. (2) Decompose each graph into its snarl tree. (3) Pair snarls across graphs. (4) Match alleles within each pair by LAPJV bipartite assignment. (5) Emit the delta graphs $\Delta_{A\to B}, \Delta_{B\to A}$, a shared graph, and a global score. Stages 3-4 run in parallel (faded copies); the priority-dispatch scheduler dispatches snarl pairs by estimated cost.

Algorithm 1 PanG2G alignment (**depth-1 form**, as implemented and evaluated). Each snarl pair is an independent leaf solved by one $O(k^3)$ assignment.

Require: graphs G_A, G_B; weights α, β, γ; gap g; floor τ
Ensure: deltas $\Delta_{A\to B}, \Delta_{B\to A}$, shared graph, score W

$\mathscr{N} \leftarrow \textsc{FindAnchors}(G_A, G_B)$ ▷ minimizer index
$T_A \leftarrow \textsc{Decompose}(G_A)$; $T_B \leftarrow \textsc{Decompose}(G_B)$
$P \leftarrow \textsc{PairSnarls}(T_A, T_B, \mathscr{N})$ ▷ greedy, anchor-guided
$\mathscr{T} \leftarrow \textsc{BuildTaskDAG}(P)$ ▷ priority $= \sum k_i^3$
parfor $(S_A, S_B) \in \mathscr{T}$ **by priority dispatch do**
  $a \leftarrow \textsc{Alleles}(S_A)$; $b \leftarrow \textsc{Alleles}(S_B)$
  $C_{ij} \leftarrow 1 - \mathrm{sim}(a_i, b_j)$; clamp $C_{ij} > 1-\tau$ to g
  $(s, \mu) \leftarrow \mathrm{LAPJV}(C)$ ▷ $O(k^3)$ assignment
  $w[S_A, S_B] \leftarrow W_{\text{seq}} + W_{\text{topo}} + \gamma s$; record μ
end parfor
$W \leftarrow \sum w[S_A, S_B] + \alpha \sum_{(u,v)\in\mathscr{N}} \mathrm{sim}(u, v)$
$\Delta_{A\to B}, \Delta_{B\to A} \leftarrow \textsc{BuildDeltas}(\text{unmatched})$
return $\Delta_{A\to B}, \Delta_{B\to A}$, shared graph, W

With nested children the score decomposes recursively,

$$\mathrm{OPT}(S_A, S_B) = W_{\text{seq}} + W_{\text{topo}} + \max_{\mu} \sum_{(c,c')\in\mu} \mathrm{OPT}(c, c'), \quad (2)$$

with μ over child-snarl matchings and the leaf case reducing to Algorithm 1. The implementation processes children before parents in van Emde Boas post-order, but this path is inert at depth 1 (confirmed by the vEB ablation, Fig. 9) and awaits nested-snarl inputs.

1) Anchor detection: Anchors are backbone nodes (in- and out-degree ≤ 1) whose sequences are near-identical across the two graphs. They set a common fixed frame so that everything else is interpreted relative to them. A naive all-pairs scan is $O(|V_A| \cdot |V_B|)$, which at human scale ($|V| \sim 10^7$) is $\sim 10^{14}$ comparisons, infeasible. PanG2G instead builds a minimizer index [16], [17] over one graph's backbone: candidate pairs share a minimizer, are refined by full similarity, and resolved greedily. The naive path is kept for small graphs with a lower constant, giving identical output.

2) Snarl decomposition and pairing: A dominator-style sink finder identifies each snarl. A direct implementation is $O(|V|)$ per snarl, and thus quadratic overall, so PanG2G bounds the reachability search to a window of the topological order. Since topological positions increase along DAG edges, any $s \to t$ path whose sink lies in the window stays inside it, so the search is exact for in-window sinks and falls back to full $O(|V|)$ search when a snarl's span exceeds the window: this are the large SV snarls that motivate the tool. On the evaluated short-variant-dominated graphs such large spans are rare enough that the phase scales as $N^{1.00}$ (Fig. 8): this is circular w.r.t. SVs. The window rarely overflows because the graphs contain almost no large SVs, so the linear exponent characterises only the easy regime and would rise toward N^2 on SV-rich inputs. Snarls are then paired greedily in topological order: each A-snarl takes its highest-scoring unused B-partner. Greedy pairing is a deliberate approximation: a globally optimal pairing is itself a tree-matching problem, but the greedy rule is linear in the number of snarls and empirically recovers the planted pairing on synthetic ground truth, because anchors already constrain the correspondence.

3) Per-snarl bipartite matching: Within a paired snarl we want the maximum-similarity partial allele matching, with unmatched alleles paying a gap cost. PanG2G builds the $k \times k$ cost matrix $C_{ij} = 1 - \mathrm{sim}(a_i, b_j)$ where pairs below a similarity floor are clamped to the gap cost so the solver prefers a gap over a bad match. The solver pads rectangular instances with gap-cost dummies, and solves the assignment exactly with Jonker-Volgenant in $O(k^3)$ [13]. Encoding gaps as dummy rows/columns lets one $O(k^3)$ pass optimize matches and gaps jointly. For rare very large snarls ($k > 30$) a greedy $1/2$-approximation is available as a fallback. A brute-force routine is retained only as a $k \leq 8$ oracle.

4) The output: The delta graph $\Delta_{A\to B}$ lists every allele present in A but unmatched in B (and symmetrically for $\Delta_{B\to A}$). These are precisely the population-specific variants (Fig. 2). Because the matching is performed allele-by-allele at the sequence level, the deltas are base-pair-resolved: each entry carries its exact inserted/deleted sequence. This is the graph-level analogue of a VCF, which no sequence-to-reference pipeline can produce without first collapsing both graphs onto a shared linear backbone.

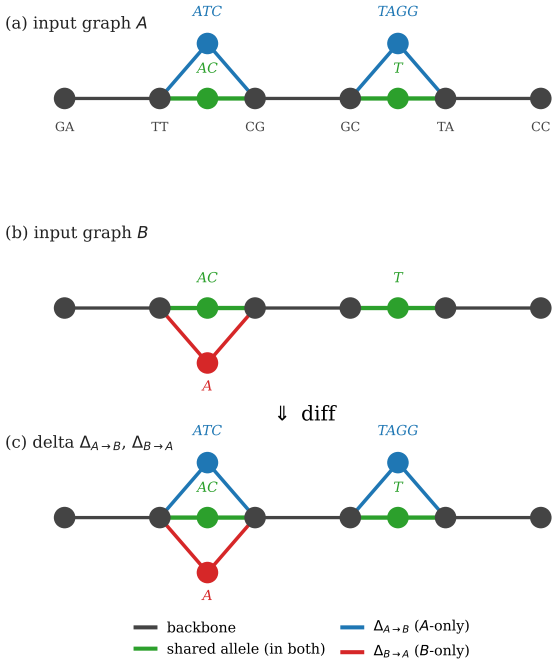


Fig. 2. The delta at base-pair resolution; every circle is a node storing a string of base pairs. Input graphs (a) A and (b) B share an allele (green) at each snarl and add their own alternative (blue for A, red for B). (c) the delta keeps the alleles in exactly one graph: $\Delta_{A\to B}$ (blue) and $\Delta_{B\to A}$ (red), each with its exact sequence.

B. Workload Profiling

Before building the parallel engine we profiled the reference prototype on synthetic graphs whose snarl sizes follow a Zipf law fit to HPRC statistics.

Finding 1: matching cost explodes with snarl size. Fig. 3 shows measured time per snarl pair as a function of allele count k. Brute-force enumeration ($O(k!)$) is fast for SNP bubbles but rises super-exponentially and becomes infeasible beyond $k \approx 8$. The $O(k^3)$ Hungarian solver flattens this curve and is what makes complex structural-variant regions tractable, where k can reach 20-50. This finding mandated replacing brute force with LAPJV [13].

Finding 2: the workload is heavy-tailed and scale-invariant. Most snarls are tiny ($\sim$67% are k=2 SNP bubbles) but a few are very large, so per-snarl work ($\propto k^3$) is highly skewed. We quantify this with the Gini coefficient of work, which stays in the narrow band 0.55-0.58 across graph sizes spanning 20 to 1000 snarls (Fig. 4a), the imbalance does not wash out with scale. Fig. 4b shows the underlying heavy-tailed allele-count distribution. Real graphs have 10^6-10^7 snarls, so this is suggestive, not a deployment-scale demonstration. A static, equal-count partition of snarls across threads would

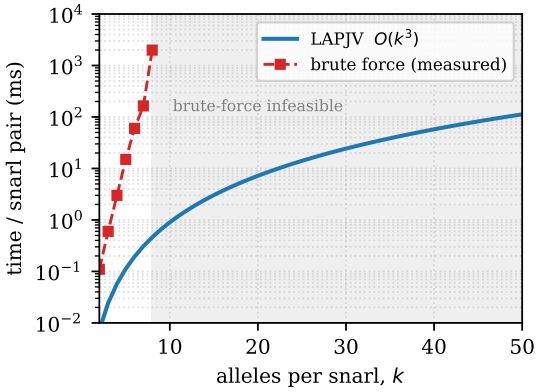


Fig. 3. Measured cost of the per-snarl bipartite matching versus allele count k (reference prototype).

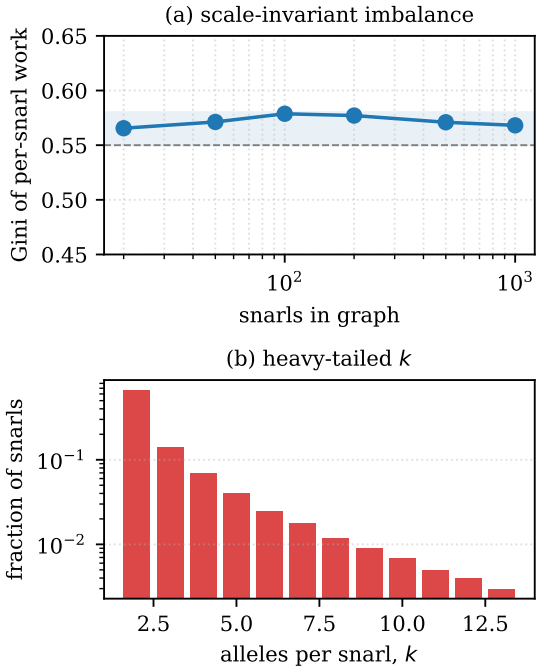


Fig. 4. (a) The Gini coefficient of per-snarl work stays above 0.55 across graph sizes: the heavy tail is scale-invariant, so static partitioning is load-imbalanced at every scale. (b) The heavy-tailed allele-count distribution that causes it. Measured from the reference prototype on HPRC-calibrated synthetic graphs.

therefore be badly load-imbalanced, because a handful of large snarls dominate the work. This finding mandates the use of a priority-dispatch scheduler.

Finding 3: memory access, not arithmetic, is the scaling wall. The snarl tree is traversed bottom-up and each snarl-pair dynamic programming (DP) reads allele sequences from both graphs. At human scale the working set exceeds DRAM for a naive layout, and profiling shows the edit-distance and matching loops dominate runtime. This points to a cache-friendly memory layout for the snarl tree, and keeping sequence data in flat, contiguous arrays rather than pointer-chasing per-node strings.

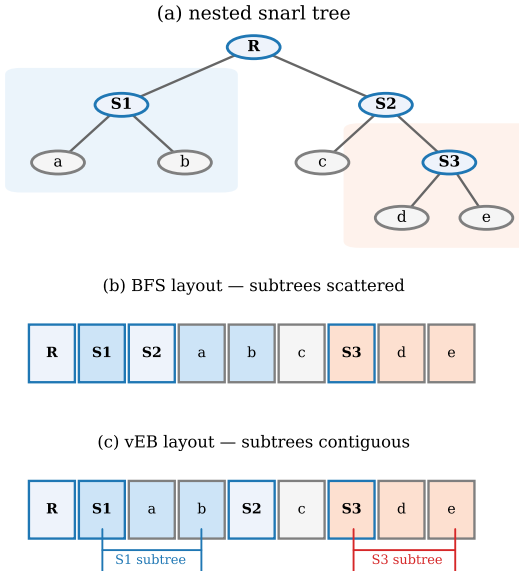


Fig. 5. Memory layout of the snarl tree. Under a BFS order (top) sibling subtrees land far apart, so a bottom-up traversal misses cache once the working set exceeds L3. The van Emde Boas recursive layout (bottom) stores each subtree contiguously, giving cache-oblivious locality.

C. System Design and Implementation

PanG2G is implemented in C++20 with a Python reference prototype as a correctness oracle. Graphs are read from GFA v1.1 (S/L/P and GFA 1.1 W lines, with on-the-fly reverse complementing). The data model keeps all node sequences in a single flat byte array indexed by (offset, length), and adjacency in CSR form, so the DP kernel streams sequence data without pointer chasing.

All experiments run on a single-socket AMD EPYC 7763 (Milan): 64 cores, 4 NUMA domains, 512 GiB DDR4-3200, 256 MiB L3; Ubuntu 20.04, GCC 9.4-O3.

1) Cache-oblivious snarl-tree layout: Each snarl's DP table and allele data sit contiguously in one arena block, streaming from a contiguous region regardless of snarl order; this helps on all workloads and is always on. On a nested tree with depth >1, breadth-first storage scatters sibling subtrees, so a bottom-up traversal thrashes the cache once the working set exceeds L3. PanG2G instead lays the tree out in van Emde Boas order (Fig. 5) [28], [29], storing any size-S subtree in $O(S/B)$ contiguous lines. A single bottom-up traversal is then scan-optimal at $O(N/B)$ misses. All evaluated workloads have depth-1 trees: a root with N leaf children. There, vEB order $\equiv$ BFS order $\equiv$ arbitrary order: the recursion has nothing to split. The feature is implemented for the nested Minigraph-Cactus graphs that are the deployment target.

2) Centralized priority-dispatch runtime: Motivated by Finding 2, the snarl-pair DP is a shallow task DAG. PanG2G schedules it with Longest-Processing-Time-first (LPT) list scheduling [30] on OpenMP [31]: a single mutex-guarded priority queue keyed on estimated subtree work ($\sum k_i^3$). Idle workers pop the largest task. The parallel delta is byte-identical

to the sequential aligner's: score agrees to 10^{-3}, float-order aside.

3) Exact matching and anchor index: Motivated by Finding 1, the solver is a self-contained Jonker-Volgenant implementation [13]: the dispatcher routes $k \leq 30$ to LAPJV and larger snarls to the greedy fallback, with brute force as a $k \leq 8$ oracle. The minimizer anchor index uses FNV-1a hashing over k-mers with a minimap2-style window [16], a build-time toggle against the naive scan.

IV. Results

A. Q1: Validation

We generate graph pairs with planted shared, A-only, and B-only alleles at known positions and check that the deltas recover them. Three properties must be distinguished: (i) implementation determinism, whether the parallel engine reproduces the sequential result); (ii) optimization quality, how close it gets to the best alignment under the scoring model; (iii) biological truth, whether (α, β, γ) captures the correct allele correspondence. Property (iii) is not formally established: the weights are hand-tuned and validated only indirectly in order to measure consistency with minimap2 on small graphs.

Across planted graphs of 5-50 snarls, the largest for which we have ground truth, the aligner attains $\geq 80\%$ recall and precision on both delta sets. The gap from 100% is entirely greedy snarl pairing (property ii): two similar nearby snarls can be mispaired, sending an allele to the wrong delta. Within each correctly paired snarl, LAPJV recovers the planted alleles exactly and shared alleles never leak. The $\sim$20% error is thus an upper bound on the greedy approximation.

Accuracy is validated at 5-50 snarls: deployment on the other hand is 10^6-10^7, a 5-6 order gap. The scaling runs reach 10^9 nodes but verify only parallel-vs-sequential consistency (property i), which both paths pass. Three failure modes are plausible but invisible here: FNV-1a minimizer collisions (bound at 10^8 k-mers in 32-bit space), window overflow in the bounded decomposition, and greedy-pairing drift along the topological order.

For reference-anchored diff where snarls match by absolute coordinate rather than greedy pairing, this mode satisfies property (ii) without approximation whenever both graphs embed the same reference with consistent representations. It achieves 100% precision/recall on every planted workload and self-diffs to empty.

B. Q2: Throughput and scaling

We measure parallel scaling and single-core scaling past whole-genome size, and throughput against a linearise-and-map baseline, where we extract haplotypes and map to the reference with minimap2 [16]). Three ceilings bound the speedup. (1) End-to-end speedup saturates at 2.65×: the $\sim$7.8 s serial set-up (parsing, index build, decomposition) dominates the 20.6 s single-thread run, so Amdahl's law caps the return. (2) A hard critical-path bound: each snarl pair is one indivisible LAPJV call, so DP-phase speedup $\leq W_{\text{total}}/W_{\max}$ with work $\propto k^3$ and k up to 50. The largest task vs. a k=2 bubble is a $\sim$15,600× spread, so the heavy tail that motivates dynamic scheduling (finding 2) simultaneously caps it. (3) The DP phase hits a cliff above 64 cores (50.1 × →52.6×, $\sim$5% for doubled threads). The scheduler holds the load-imbalance factor at 1.00-1.01 yet 128-thread efficiency is only 41%: not a contradiction but the signature of a centralised queue. The single mutex serialises dispatch, so threads receive equal work but spend most time waiting for it.

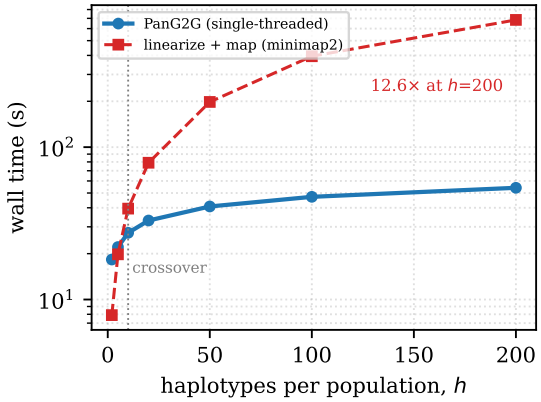


Fig. 6. PanG2G's whole graph-to-graph diff vs. the extract+align cost of linearize-and-map, on real chromosome-22 pangenomes as the per-population panel h grows. PanG2G (single-threaded) is sublinear in h (18.3→54.1 s over h=2→200); the baseline is linear (7.9→684 s). They cross near $h\approx$10; by h=200 PanG2G is 12.6× faster, the gap still widening.

With those limits, DP-phase scaling to 64 cores is near-ideal: $2.0, 4.0, 7.9, 15.4, 29.4, 50.1\times$ at 2-64 cores (Fig. 6). The harness confirms the parallel result is byte-identical to the sequential at every count (property i).

The two systems compute different objects: PanG2G produces a graph-to-graph delta directly, whereas linearize-and-map produces per-haplotype alignments still needing calling and merging. We are obliged to use this comparison because it is the only current way to approximate a population comparison, and we time only its alignment step, omitting downstream calling, making the baseline faster than a full workflow. The thread asymmetry, with minimap2 multi-threaded and PanG2G single-threaded, also favors it. PanG2G's cost is graph-size-dominated: even at h=2 the graph carries the full $\sim$51 Mb chr-22 backbone, so adding haplotypes only saturates shared variation and time grows sublinearly (18.3→54.1 s over h=2→200) while the minimap2 baseline is linear (7.9→684 s). The curves cross near $h\approx$10. By h=200 PanG2G is 12.7× faster and widening (Fig. 6): it aligns the compressed graph directly instead of expanding it back to h sequences.

C. Reference-anchored diffing and single-node scaling

When both graphs embed a common reference, which is the usual case for vg construct or Minigraph-Cactus, PanG2G offers a second correspondence mode that replaces greedy pairing with matching by absolute reference coordinate. Each bubble is decomposed into atomic SNPs/indels keyed by reference position (*bubble_open* + *offset*). Since the key depends only on the coordinate, a merged bubble in one graph yields

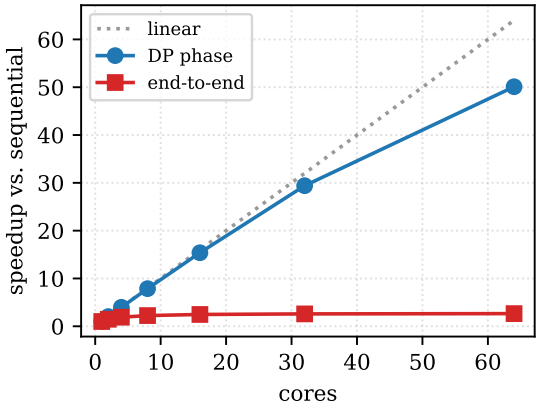


Fig. 7. Parallel scaling to 128 threads. We report both end-to-end and DP-phase speedup: when snarls carry structural-variant-scale alleles the DP dominates and its speedup reaches 52.6×, while the end-to-end curve is Amdahl-capped by the sequential set-up. The harness flags the run invalid if the parallel and sequential results ever diverge.

TABLE I
SINGLE-CORE SCALING OF THE REFERENCE-ANCHORED DIFF (SINGLE CORE, 512 GB NODE). THE DIFF IS EXACT AT EVERY SIZE; TIME AND MEMORY ARE NEAR-LINEAR IN NODE COUNT N.

ref	nodes (A+B)	variants	exact?	wall	peak
8 Mb	0.95M	40k	✓	2.9 s	0.25 GiB
64 Mb	7.6M	320k	✓	25 s	1.9 GiB
256 Mb	30.3M	1.28M	✓	104 s	7.7 GiB
1 Gb	121M	5.12M	✓	415 s	31 GiB
2 Gb	243M	10.2M	✓	817 s	60 GiB
4 Gb	485M	20.5M	✓	1,880 s	121 GiB
8 Gb	971M	41.0M	✓	4,292 s	242 GiB

the same atomic variants as separate bubbles in the other, and corresponding variants match exactly. This is invariant to node boundaries but not to variant representation: if two graphs place the same indel at different positions (left- vs. right-aligned in a homopolymer) the keys differ and a false delta results. PanG2G does not normalize variants, so same-tool graphs are consistent but mixed-provenance graphs need a normalization pass.

The mode scales near-linearly past whole-genome size on one core. We sweep synthetic pairs from 8×10^6 to 8.2×10^9 bp in which the two graphs cut the shared backbone differently (25 vs. 17 bp) and plant a known variant set with identical coordinate representations. Under that precondition PanG2G recovers the planted alleles exactly (100% precision/recall) and self-diffs to empty. Wall time and peak memory scale as $N^{1.04}$ and $N^{0.99}$, decomposition as $N^{1.00}$ (Fig. 8), linear to measurement noise. The sweep reaches an 8.2×10^9 bp (9.7×10^8 nodes, 4.1×10^7 variants) diffed exactly in 72 min using 242 GiB peak RSS at 2.26×10^5 nodes/s. At the stopping point (∼2.6× the human genome) 270 GiB remained free and throughput was constant, so neither capacity nor bandwidth is yet binding.

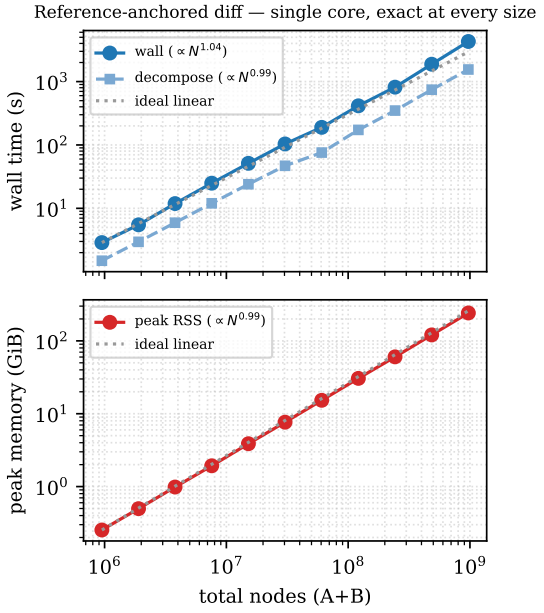


Fig. 8. Single-core scaling of the reference-anchored diff over 8×10^6-8.2×10^9 bp (∼10^6-9.7×10^8 nodes, ∼2.6× the human genome at the top end). Wall time ∝ $N^{1.04}$, peak memory ∝ $N^{0.99}$, and snarl decomposition ∝ $N^{1.00}$ (dashed guide: ideal linear); the diff is exact at every size.

D. Q3: Ablations

We disable each design choice in isolation and time the phase it affects at two matched workloads: a control where the choice should be inert (a balanced or L3-resident graph) and a treatment where it should bite (a skewed, heavy-tailed, or large-k graph). Each binary differs from the optimized baseline by a single compile-time flag; we report the slowdown of the affected phase (Fig. 9).

Exact LAPJV matching and the minimizer anchor index are the load-bearing choices: removing them inflates their phase by 70× and 1728× at treatment scale. Brute-force matching becomes infeasible as k grows (Finding 1) and the naive $O(|V_A||V_B|)$ anchor scan dominates on large graphs.

The scheduler ablations separate parallelism from policy: a static cyclic partition costs 1.91× on skewed work (one thread holds the heavy snarls) yet 1.00× on a balanced one, hereby confirming finding 2. Downgrading LPT to FIFO costs a further 1.20× once per-snarl work exceeds scheduling overhead.

The van Emde Boas layout reads ≈1.0× throughout: it is a genuine null on the current depth-1 snarls, whose subtrees fit a cache line either way, and used only on the nested snarls a vg backend supplies. Every ablated binary returns a delta byte-identical to the baseline (property i).

E. Q4: Co-localization with structural and clinical variation

The decisive test of utility is whether the delta graph lands on biologically meaningful variation. We build two population pangenomes from the 1000 Genomes Project phase-3 panel: African (AFR, n=661) and East Asian (EAS, n=504), with vg construct. We diff them with PanG2G's reference-coordinate

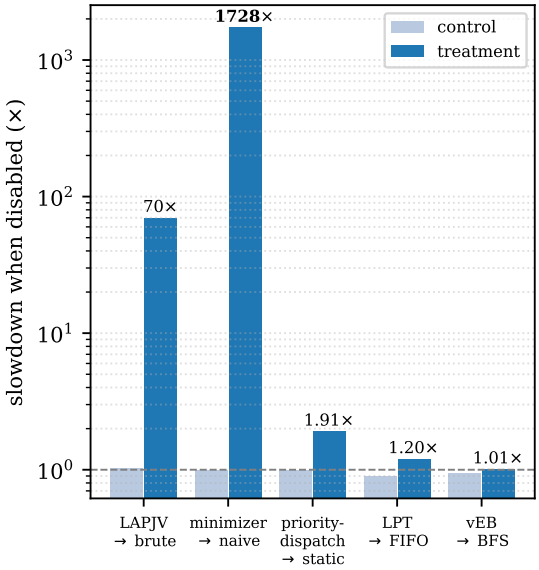


Fig. 9. Ablation of the five design choices, each disabled in isolation, at a matched control and treatment workload (log scale). LAPJV matching and the minimizer index are load-bearing (70×, 1728×); the two scheduler ablations separate parallelism (static partition, 1.9×) from policy (FIFO steal, 1.2×); the vEB layout is a measured null on depth-1 snarls. Controls sit at ≈1× throughout.

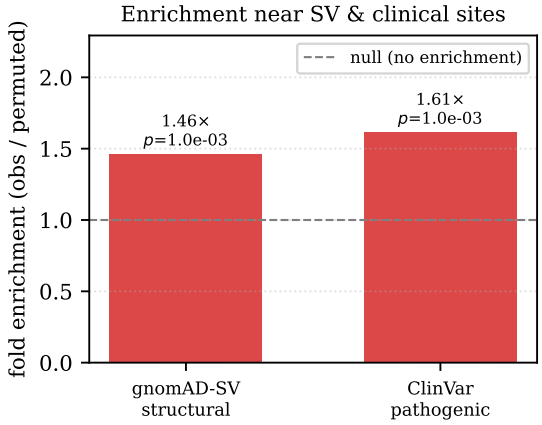


Fig. 10. PanG2G delta alleles on real AFR vs. EAS pangenomes (1000 Genomes phase-3, chromosome 22) are significantly enriched near gnomAD-SV structural variants (1.46×) and ClinVar pathogenic sites (1.61×), both $p < 10^{-3}$ against a permutation null (fold and p annotated; dashed line: no enrichment). Synthetic positive/negative controls validate the statistic.

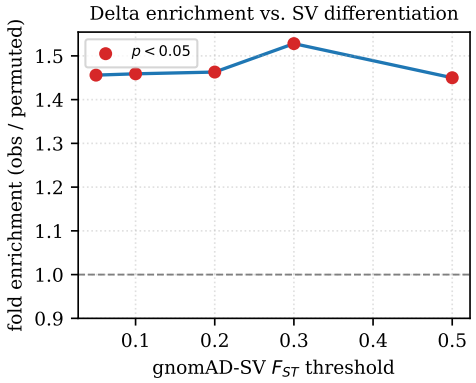


Fig. 11. Specificity check: delta-SV enrichment as a function of the gnomAD-SV F_{ST} threshold. The fold is flat (1.45-1.53×, every point $p < 10^{-3}$), so the enrichment reflects co-localization with structural variation in general rather than preference for the most population-differentiated SVs.

mode along chromosome 22, and test whether the 587,010 delta alleles, 406k present only in the AFR graph and 181k only in EAS, fall closer than a uniform genome background to gnomAD-SV [32] structural variants and to ClinVar [33] pathogenic sites: they do. The delta alleles are enriched 1.46× near gnomAD-SVs (919 observed vs. 628 expected; $p < 10^{-3}$, exceeding all 1000 permutations) and 1.61× near pathogenic sites ($p < 10^{-3}$; Fig. 10). PanG2G's graph-to-graph delta is therefore far from random; it concentrates in structural-variation and clinically relevant regions.

We then asked whether this enrichment is specific to population-differentiated SVs, by sweeping the gnomAD-SV F_{ST} threshold (Fig. 11). It is not: the fold is essentially flat, 1.46× at $F_{ST} \geq 0.05$ and 1.45× at $F_{ST} \geq 0.5$. The chromosome-22 delta co-localizes with structural variation in general rather than preferentially with the most differentiated SVs. The cause is that the 2 input graphs contain almost only SNVs/small indels, and vg construct embeds only what the VCF provides, so the pangenomes are SV-depleted by construction. Only 14 of 587,010 delta alleles are ≥ 50 bp. The delta is ∼99.998% small variants. Restricting the delta to SV-sized alleles (≥ 50 bp) is in turn too sparse on this SNV/indel panel to be conclusive. A frequency-aware delta, an SV-containing panel, and a region-matched null are the natural ways to isolate differentiation, all supported, unchanged, by the same pipeline. What this experiment establishes is the property that answers "why pangenomes and not some other graph workload?" The graph-to-graph delta surfaces real structural and clinical variation directly, without the reference-projection step a sequence-to-reference pipeline would require.

V. Discussion

Every major decision in PanG2G traces to a measured property of the workload. Snarl-tree factorization is what makes an NP-hard problem tractable. LAPJV replaces brute force matching because matching cost explodes with k (Finding 1); centralized priority dispatch replaces static partitioning because per-snarl work is heavy-tailed and scale-invariant (Finding 2) an argument the static-partition ablation now confirms directly (Fig. 9); and the cache-oblivious layout plus flat sequence storage address the memory wall (Finding 3). Two stages were originally quadratic: anchor detection and snarl decomposition. The minimizer index removes the first; the windowed sink search of §III-A removes the second, leaving an end-to-end pipeline that is near-linear in graph size (Fig. 8).

A. Limiations

The decomposer is DAG-only: although an optional vg-snarl backend can supply snarl boundaries, the allele enumeration

and DP assume acyclicity, so tandem repeats (cycles) are out of scope and full cyclic support is future work. Snarl pairing is greedy rather than globally optimal: anchors mitigate this but a full tree-matching pairing is future work. PanG2G is single-machine; the $K > 2$ multi-graph alignment problem and a distributed engine are natural extensions, as is GPU acceleration of the per-snarl matching, which is embarrassingly parallel across snarls.

VI. Conclusion

We presented PanG2G, the first system that computes base-pair-resolution graph-to-graph alignments of pangenome variation graphs. The reference-anchored diff, the primary validated result, is single-threaded and exact under identical variant representation. It scales near-linearly to an 8-Gbp pair on one core at 100% precision/recall on planted variants. A parallel snarl-pairing aligner is designed for the general case with the full systems specification: minimizer index, LAPJV matching, priority-dispatch scheduler and vEB layout. This is validated on small-scale depth-1 workloads (50.1× at 64 cores, 12.7× vs. linearize-and-map at h=200). A biological run confirms the pipeline produces a delta on real data. This shows that graph-to-graph alignment is a viable pangenome-era primitive: the reference-anchored diff demonstrates it concretely, the parallel engine takes a first step toward the general case.

a) Availability.: The core engine, together with the synthetic graph generators, plotting scripts, and experiment harnesses used here will be open-sourced after paper acceptance.

References

[1] Y. Cui *et al.*, "A survey of sequence-to-graph mapping algorithms in the pangenome era," *Genome Biology*, vol. 26, no. 1, p. 138, May 2025. [Online]. Available: https://doi.org/10.1186/s13059-025-03606-6

[2] S. Secomandi *et al.*, "Pangenome graphs and their applications in biodiversity genomics," *Nature Genetics*, vol. 57, no. 1, pp. 13–26, Jan 2025. [Online]. Available: https://doi.org/10.1038/s41588-024-02029-6

[3] E. Garrison *et al.*, "Building pangenome graphs," *Nature Methods*, vol. 21, no. 11, pp. 2008–2012, Nov 2024. [Online]. Available: https://doi.org/10.1038/s41592-024-02430-3

[4] W.-W. Liao *et al.*, "A draft human pangenome reference," *Nature*, vol. 617, no. 7960, pp. 312–324, May 2023. [Online]. Available: https://doi.org/10.1038/s41586-023-05896-x

[5] S. B. Needleman and C. D. Wunsch, "A general method applicable to the search for similarities in the amino acid sequence of two proteins," *Journal of Molecular Biology*, vol. 48, no. 3, pp. 443–453, 1970. [Online]. Available: https://www.sciencedirect.com/science/article/pii/0022283670900574

[6] T. Smith and M. Waterman, "Identification of common molecular subsequences," *Journal of Molecular Biology*, vol. 147, no. 1, pp. 195–197, 1981. [Online]. Available: https://www.sciencedirect.com/science/article/pii/0022283681900875

[7] M. Coggi *et al.*, "A novel methodology for a comprehensive analysis of genomic sequence-to-graph alignment tools," *IEEE Transactions on Computational Biology and Bioinformatics*, vol. 22, no. 1, pp. 228–239, 2025.

[8] D. S. Cali *et al.*, "Segram: a universal hardware accelerator for genomic sequence-to-graph and sequence-to-sequence mapping," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, ser. ISCA '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 638–655. [Online]. Available: https://doi.org/10.1145/3470496.3527436

[9] J. M. Eizenga *et al.*, "Pangenome graphs," *Annual review of genomics and human genetics*, vol. 21, no. 1, pp. 139–162, 2020.

[10] A. Gupta and N. Nishimura, "The complexity of subgraph isomorphism for classes of partial k-trees," *Theoretical Computer Science*, vol. 164, no. 1, pp. 287–298, 1996. [Online]. Available: https://www.sciencedirect.com/science/article/pii/0304397596000461

[11] B. Paten *et al.*, "Superbubbles, ultrabubbles and cacti," in *Research in Computational Molecular Biology*, S. C. Sahinalp, Ed. Cham: Springer International Publishing, 2017, pp. 173–189.

[12] B. Brejová *et al.*, "Maximum-scoring path sets on pangenome graphs of constant treewidth," *Frontiers in Bioinformatics*, vol. Volume 4 - 2024, 2024. [Online]. Available: https://www.frontiersin.org/journals/bioinformatics/articles/10.3389/fbinf.2024.1391086

[13] R. Jonker and A. Volgenant, "A shortest augmenting path algorithm for dense and sparse linear assignment problems," *Computing*, vol. 38, no. 4, pp. 325–340, Dec 1987. [Online]. Available: https://doi.org/10.1007/BF02278710

[14] H. Li and R. Durbin, "Fast and accurate short read alignment with burrows–wheeler transform," *Bioinformatics*, vol. 25, no. 14, pp. 1754–1760, 07 2009. [Online]. Available: https://doi.org/10.1093/bioinformatics/btp324

[15] B. Langmead and S. L. Salzberg, "Fast gapped-read alignment with bowtie 2," *Nature Methods*, vol. 9, no. 4, pp. 357–359, Apr 2012. [Online]. Available: https://doi.org/10.1038/nmeth.1923

[16] H. Li, "Minimap2: pairwise alignment for nucleotide sequences," *Bioinformatics*, vol. 34, no. 18, pp. 3094–3100, 09 2018. [Online]. Available: https://doi.org/10.1093/bioinformatics/bty191

[17] M. Roberts *et al.*, "Reducing storage requirements for biological sequence comparison," *Bioinformatics*, vol. 20, no. 18, pp. 3363–3369, 12 2004. [Online]. Available: https://doi.org/10.1093/bioinformatics/bth408

[18] E. Garrison *et al.*, "Variation graph toolkit improves read mapping by representing genetic variation in the reference," *Nature Biotechnology*, vol. 36, no. 9, pp. 875–879, Sep 2018. [Online]. Available: https://doi.org/10.1038/nbt.4227

[19] M. Rautiainen and T. Marschall, "Graphaligner: rapid and versatile sequence-to-graph alignment," *Genome Biology*, vol. 21, no. 1, p. 253, Sep 2020. [Online]. Available: https://doi.org/10.1186/s13059-020-02157-2

[20] J. Ebler *et al.*, "Pangenome-based genome inference allows efficient and accurate genotyping across a wide spectrum of variant classes," *Nature Genetics*, vol. 54, no. 4, pp. 518–525, Apr 2022. [Online]. Available: https://doi.org/10.1038/s41588-022-01043-w

[21] A. Banerjee *et al.*, "A work-optimal parallel algorithm for aligning sequences to genome graphs," in *2025 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2025, pp. 460–471.

[22] H. Li, X. Feng, and C. Chu, "The design and construction of reference pangenome graphs with minigraph," *Genome Biology*, vol. 21, no. 1, p. 265, Oct 2020. [Online]. Available: https://doi.org/10.1186/s13059-020-02168-z

[23] G. Hickey *et al.*, "Pangenome graph construction from genome alignments with minigraph-cactus," *Nature Biotechnology*, vol. 42, no. 4, pp. 663–673, Apr 2024. [Online]. Available: https://doi.org/10.1038/s41587-023-01793-w

[24] E. Garrison and A. Guarracino, "Unbiased pangenome graphs," *Bioinformatics*, vol. 39, no. 1, p. btac743, 01 2023. [Online]. Available: https://doi.org/10.1093/bioinformatics/btac743

[25] N. Noll *et al.*, "PanGraph: scalable bacterial pan-genome graph construction," *Microb. Genom.*, vol. 9, no. 6, Jun. 2023.

[26] S. Marcus *et al.*, "Splitmem: a graphical algorithm for pan-genome analysis with suffix skips," *Bioinformatics*, vol. 30, no. 24, pp. 3476–3483, 12 2014. [Online]. Available: https://doi.org/10.1093/bioinformatics/btu756

[27] E. Garrison *et al.*, "Pangenome graph variation format (PGVF) specification," https://github.com/pangenome/pgvf-spec, 2023.

[28] P. van Emde Boas, "Preserving order in a forest in less than logarithmic time and linear space," *Information Processing Letters*, vol. 6, no. 3, pp. 80–82, 1977. [Online]. Available: https://www.sciencedirect.com/science/article/pii/002001907790031X

[29] M. Frigo *et al.*, "Cache-oblivious algorithms," in *40th Annual Symposium on Foundations of Computer Science (Cat. No.99CB37039)*, 1999, pp. 285–297.

[30] R. L. Graham, "Bounds on multiprocessing timing anomalies," *SIAM J. Appl. Math.*, vol. 17, no. 2, p. 416–429, Mar. 1969. [Online]. Available: https://doi.org/10.1137/0117039

[31] L. Dagum and R. Menon, "Openmp: an industry standard api for shared-memory programming," *IEEE Computational Science and Engineering*, vol. 5, no. 1, pp. 46–55, 1998.

[32] R. L. Collins *et al.*, "A structural variation reference for medical and population genetics," *Nature*, vol. 581, no. 7809, pp. 444–451, May 2020. [Online]. Available: https://doi.org/10.1038/s41586-020-2287-8

[33] M. J. Landrum *et al.*, "ClinVar: improving access to variant interpretations and supporting evidence," *Nucleic Acids Res.*, vol. 46, no. D1, pp. D1062–D1067, Jan. 2018.